

Rapport des TPs Moteur3D

Amadou DIARRA

Année universitaire 2024–2025

1 Introduction

L'objectif de ce projet était de saisir le fonctionnement d'OpenGL. Pour atteindre cette compréhension, il a été nécessaire d'approfondir la réalisation des travaux pratiques.

2 TP1

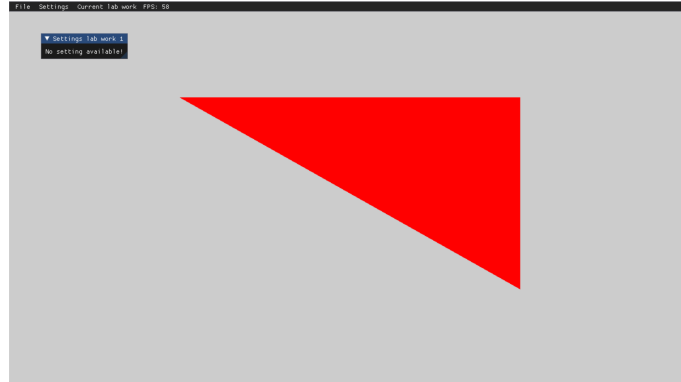


FIGURE 1 – TP1 : premier rendu d'un triangle rouge en 2D

L'objectif de ce travail pratique était de s'initier à OpenGL. Il consistait à comprendre comment utiliser un VBO et un VAO, ainsi que la liaison avec le fragment shader et le vertex. Une fois ces concepts compris en suivant les explications, afficher un triangle devint une tâche simple.

3 TP2

L'objectif de ce travail pratique consistait à améliorer le programme du TP1 en passant d'un simple triangle à un rectangle en mouvement. Pour ce faire, il a

été nécessaire d'ajouter davantage de points dans le VBO (Vertex Buffer Object) et d'introduire un EBO (Element Buffer Object). L'utilisation de l'EBO permettait de spécifier les indices des points pour définir les triangles. Par exemple, en utilisant les indices 1, 2, 3, 4, 2, 0, on pouvait tracer un triangle avec les points 1, 2, 3, et un autre avec les points 4, 2, 0.

Une autre étape importante était de comprendre le système d'uniforme, qui consiste à envoyer des informations aux shaders, en l'occurrence aux vertex shaders dans ce contexte. Cela était nécessaire pour effectuer une translation, c'est-à-dire déplacer le rectangle. En résumé, l'objectif était d'enrichir le programme en ajoutant des fonctionnalités telles que la gestion d'un rectangle mobile, l'utilisation d'un EBO pour définir les triangles, et la transmission d'informations via des uniforms pour réaliser des transformations comme la translation.

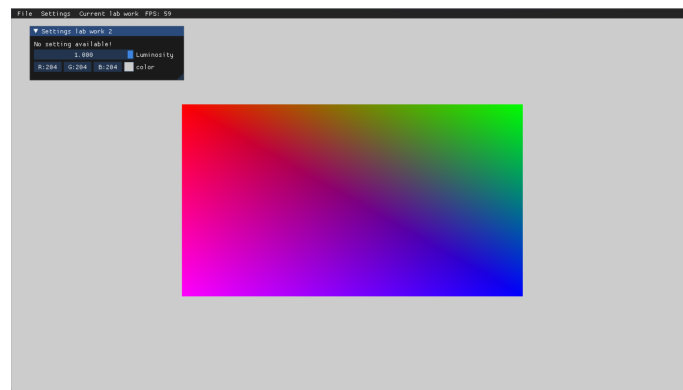


FIGURE 2 – TP2 : dégradé coloré sur un rectangle

TP3 — Passage à la 3D et gestion des caméras

Factorisation et réutilisabilité via ShaderUtils À partir du TP3, une réflexion a été menée pour améliorer la clarté et la maintenabilité du code, notamment dans la gestion des shaders. En effet, la création et la compilation des shaders (vertex et fragment), ainsi que la gestion des erreurs associées, étaient répétées à l'identique dans chaque TP. Pour éviter cette redondance et réduire le risque d'erreur, une classe utilitaire nommée `ShaderUtils` a été introduite.

Cette classe propose trois fonctions principales : `compileShader`, `createProgramFromSources`, et `createProgramFromFiles`. Ces méthodes encapsulent la logique de compilation des shaders, de création des programmes OpenGL, ainsi que la gestion des logs d'erreurs. En centralisant ces opérations, nous avons pu simplifier les fonctions d'initialisation de chaque TP, rendant le code plus lisible et modulaire. Par exemple, dans le TP3, l'appel à `createProgramFromFiles` permet de charger et compiler les shaders GLSL du modèle de Phong en une ligne, au lieu d'une vingtaine précédemment.

Ce choix de factorisation témoigne d'une volonté d'appliquer de bonnes pratiques logicielles, en particulier la séparation des responsabilités et la réutilisabilité du code. Cela s'est révélé particulièrement utile au fur et à mesure de l'ajout de complexité dans les shaders (gestion des textures, illumination avancée, etc.), et a

permis de se concentrer sur les logiques de rendu plutôt que sur la gestion bas-niveau d'OpenGL.

Ce TP marque une transition majeure dans le projet : on quitte la 2D pour explorer la 3D, en introduisant une gestion complète des transformations géométriques via les matrices **Model**, **View** et **Projection**. L'objectif principal était d'afficher un cube dans l'espace 3D, animé en rotation, tout en manipulant la caméra pour explorer la scène.

La caméra a donc été renforcée avec la possibilité de générer dynamiquement les matrices **View** (calculées à partir de la position et de la direction de la caméra) et **Projection** (perspective, contrôlée via un champ de vision). La position et la direction de la caméra sont mises à jour par interaction clavier (déplacement dans les axes) et souris (rotation du regard).

D'un point de vue technique, la gestion des entrées utilisateur (**SDL_Event**) permet de modifier la position et la direction de la caméra à chaque frame, et la matrice **Model** est modifiée continuellement en fonction du temps (**p_deltaTime**) pour animer le cube.

Les matrices sont ensuite envoyées aux shaders (**lw3.vert**, **lw3.frag**) via des variables **uniform**, afin d'appliquer correctement les transformations et obtenir un rendu réaliste. Le rendu s'appuie sur une géométrie de cube définie manuellement (VBO, EBO, VAO) et visualisée grâce à une projection en perspective.

Ce TP nous a permis de mieux comprendre la chaîne de transformation en 3D et la nécessité de structurer le code pour le rendre plus modulaire (séparation des responsabilités entre **init**, **render**, **animate**, **handleEvents**, etc.). Le tout a été intégré dans une interface graphique (ImGui) pour ajuster des paramètres dynamiquement, comme la vitesse de la caméra ou le champ de vision.

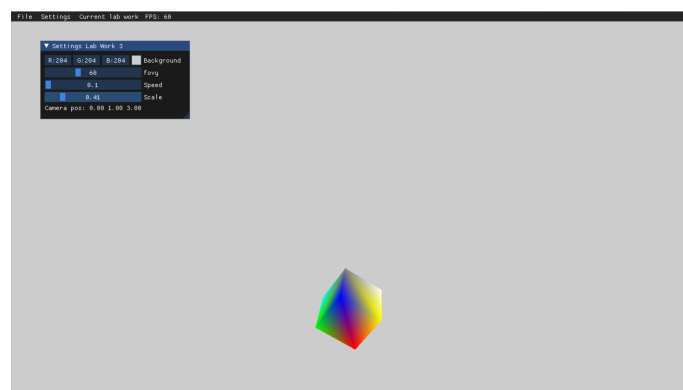


FIGURE 3 – TP3 : cube 3D tournant avec interface ImGui

4 TP4 — Calcul d'éclairage local avec le modèle de Phong

Ce TP introduit le rendu réaliste d'un objet 3D via le calcul d'éclairage local, en mettant en œuvre le modèle d'illumination de Phong. Le but était de simuler l'effet d'une lumière ponctuelle (placée à l'origine du repère en *View Space*, donc sur la caméra) sur un maillage complexe, à travers trois composantes : ambiante, diffuse et spéculaire. Tous les calculs ont été réalisés en espace caméra, ce qui implique de transformer correctement les normales et les vecteurs de direction.

Sur le plan technique, des matrices supplémentaires ont été calculées dans le code C++ : la matrice modèle-vue (`MVMatrix`) pour positionner l'objet dans l'espace caméra, la matrice normale (`NormalMatrix`) pour transformer correctement les normales, et la position de la caméra, transmise également en *View Space*. Ces données ont été envoyées au shader, où les calculs d'illumination ont été effectués avec soin pour tenir compte des propriétés matérielles de l'objet (couleurs, brillance, textures éventuelles).

Le modèle 3D utilisé (le *bunny* ou la scène *conference*) a été chargé via **Assimp** et affiché à l'aide de la classe `TriangleMesh`, conçue pour gérer les buffers OpenGL (VAO/VBO/EBO) et les matériaux. Une interface via **ImGui** permettait de contrôler certains paramètres comme la vitesse de déplacement ou le champ de vision, mais aussi d'observer l'effet des différentes composantes lumineuses.

Ce TP a représenté un point clé dans la compréhension de la pipeline graphique programmable, en particulier la séparation des rôles entre le CPU (préparation des données) et le GPU (calcul des effets visuels). La difficulté principale résidait dans la transformation cohérente des vecteurs en espace caméra et la bonne transmission des variables uniformes aux shaders.

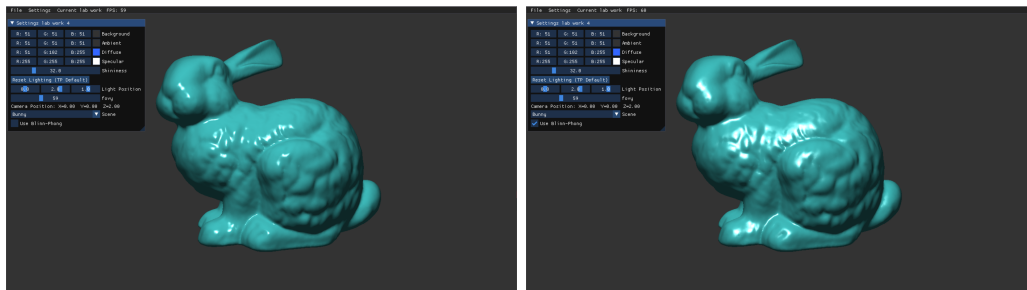


FIGURE 4 – Comparaison entre Phong (gauche) et Blinn-Phong (droite)

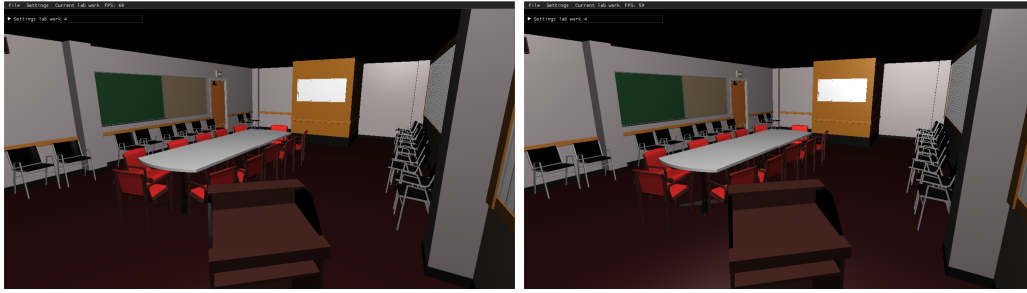


FIGURE 5 – Composantes spéculaires : Phong (gauche) vs Blinn (droite)

TP5

Ce rapport synthétise l'ensemble du travail réalisé lors du TP5, consacré à la gestion des textures et à l'amélioration du rendu dans un moteur 3D temps réel avec OpenGL. L'objectif principal était de mettre en place un système flexible et performant capable de gérer des matériaux enrichis par différentes cartes de textures (diffuse, spéculaire, normale, shininess) tout en assurant un rendu fidèle sur des scènes complexes comme *Sponza*. L'éclairage a été géré selon les modèles de Phong et Blinn-Phong, avec un calcul temps réel des interactions lumière-matière dans les shaders GLSL. Pour améliorer le réalisme visuel, nous avons intégré l'usage de cartes de normales en espace tangent, impliquant la construction correcte de la matrice TBN dans le vertex shader.

Une interface ImGui a été intégrée afin de permettre une interaction dynamique avec les paramètres de rendu : éclairage, matériau, modèle d'éclairage, champ de vision ou encore choix de la scène à afficher. L'utilisateur peut également recharger les shaders à chaud, ce qui a facilité le cycle de développement. Le rendu d'arrière-plan a été enrichi par l'ajout d'une skybox, chargée via une cubemap personnalisée, sans dépendance externe.

Ce TP nous a confrontés à plusieurs défis techniques : compatibilité des shaders avec ou sans textures, gestion correcte des transformations pour des scènes de grande envergure, et paramétrage intuitif de la caméra. Nous avons porté une attention particulière à la modularité et à la réutilisabilité du code, afin de préparer de futures extensions comme le shading différé ou le PBR.

En somme, cette étape a permis de renforcer notre compréhension du pipeline graphique moderne, tout en structurant proprement notre moteur pour intégrer progressivement des effets plus avancés.



FIGURE 6 – TP5 : rendu global de la scène Sponza avec textures



FIGURE 7 – TP5 : gros plan sur les détails de la texture et normal map



FIGURE 8 – TP5 : transparence et filtrage dans la scène

5 TP6 — Rendu différé (Deferred Shading)

Ce TP avait pour objectif la mise en œuvre d'une technique de rendu avancée largement utilisée dans les moteurs 3D : le *deferred shading* (rendu différé). Contrairement au rendu direct (*forward rendering*), cette approche sépare le traitement géométrique du calcul d'éclairage afin de mieux gérer la complexité des scènes.

Nous avons développé un pipeline complet structuré en deux passes distinctes :

- **Geometry pass** : enregistrement des données géométriques (position, normale, etc.) dans un *G-Buffer*.
- **Shading pass** : calcul final de la lumière en lisant les données du G-Buffer.

Le projet exploite des Framebuffer Objects (FBOs), le mécanisme des Multiple Render Targets (MRT) et une interface graphique ImGui pour le débogage et la visualisation du contenu des buffers.

5.1 Effets mis en place

- Construction d'un G-Buffer contenant six textures : position, normale, couleur ambiante, diffuse, spéculaire + brillance, et profondeur.
- Séparation du rendu en deux passes : géométrie puis éclairage.
- Implémentation du modèle Blinn-Phong pour le shading différé.
- Affichage interactif des textures du G-Buffer via ImGui.
- Utilisation d'un quad plein écran pour la passe d'éclairage.
- Gestion dynamique de la position de la lumière à travers l'interface.

5.2 Architecture technique

5.2.1 G-Buffer et FBO

Chaque information du G-Buffer est stockée dans une texture GPU au format `GL_RGBA16F` (pour les couleurs/vecteurs) ou `GL_DEPTH_COMPONENT` (pour la profondeur). Ces textures sont attachées à un FBO via `glFramebufferTexture2D`.

5.2.2 Geometry Pass

Cette première passe enregistre les données par fragment dans les textures du G-Buffer :

- `gPosition` : position du fragment dans l'espace monde.
- `gNormal` : normale transformée.
- `gAmbient`, `gDiffuse`, `gSpecular` : propriétés de matériau.

5.2.3 Shading Pass

Le shading final est effectué sur un quad plein écran, en lisant les informations du G-Buffer via un shader Blinn-Phong. La lumière est configurée dynamiquement via uniformes.

5.2.4 Interface Utilisateur

L'interface graphique ImGui permet :

- d'activer ou désactiver l'affichage des textures du G-Buffer,
- de sélectionner dynamiquement la texture à afficher,
- de contrôler la position de la lumière,
- d'afficher un aperçu des textures (`ImGui::Image`).

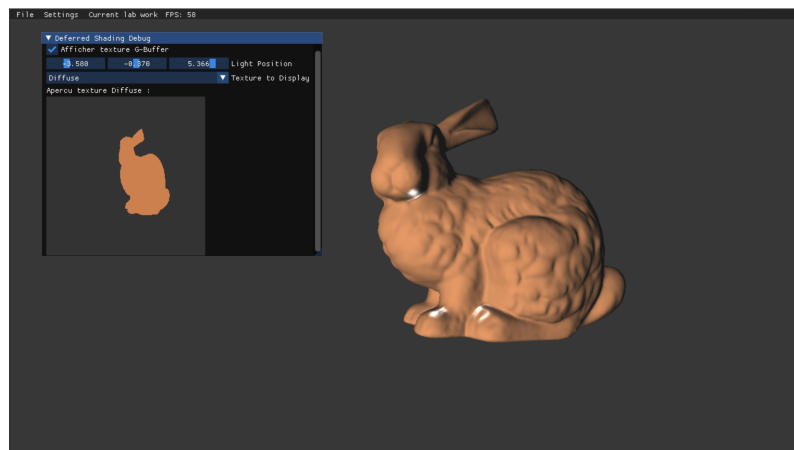


FIGURE 9 – Affichage de la texture diffuse à partir du G-Buffer

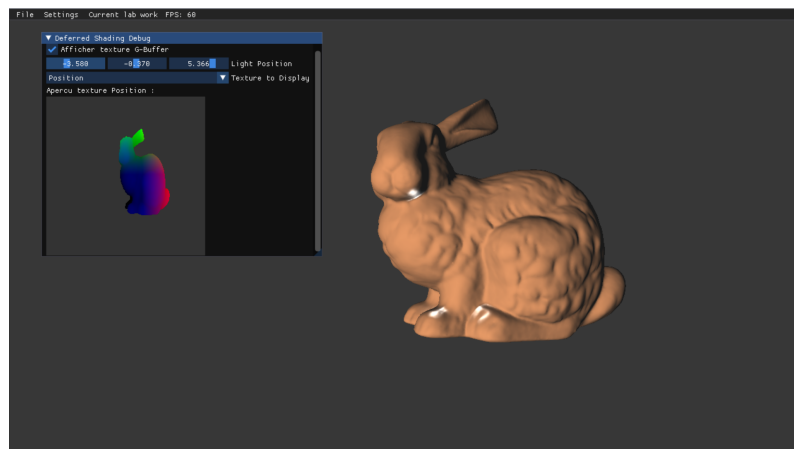


FIGURE 10 – Affichage de la texture de position encodée en RGB

L'utilisation d'ImGui s'est avérée très pratique pour explorer visuellement le contenu du G-Buffer, facilitant le débogage et l'analyse du pipeline graphique.

6 Difficultés rencontrées

J'ai rencontré diverses difficultés tout au long des travaux pratiques, tant sur le plan technique que conceptuel. La montée en complexité progressive a exigé une

bonne maîtrise d'OpenGL moderne (version 4.5), notamment dans la gestion des buffers (VAO, VBO, EBO), des shaders et des différentes transformations d'espace (Model, View, Projection). Un exemple concret issu du TP4 sur l'éclairage local illustre bien ces défis : lors du rendu de la scène de conférence, certaines parties du modèle apparaissaient totalement sombres. Après analyse, le problème venait de normales mal orientées, qu'il a fallu corriger dynamiquement en les inversant dans le shader lorsqu'elles ne pointaient pas vers la caméra. Cette étape a mis en lumière l'importance de bien comprendre le passage dans le *View Space* ainsi que le fonctionnement du modèle de Phong. À cela s'ajoutaient d'autres difficultés plus techniques, comme des erreurs de liaison GLSL (par exemple, l'oubli de `fragUV` dans le vertex shader), des shaders incomplets provoquant des erreurs de type `premature EOF`, ou encore des problèmes de compatibilité clavier selon la disposition utilisée (QWERTY/AZERTY). L'ordre incorrect des textures dans le G-Buffer a également nécessité l'emploi de couleurs de test pour les valider visuellement. Enfin, des aspects plus avancés comme l'intégration de textures conditionnelles, la gestion du pipeline de *deferred shading* ou encore la mise en place d'interfaces interactives avec ImGui ont demandé une organisation soignée, un débogage rigoureux et une bonne autonomie pour surmonter les obstacles, souvent liés à des erreurs peu explicites.

Références bibliographiques

1. **Cours M3D**
Supports de cours fournis dans le cadre du module de Moteur 3D (M1 ISICG – Université de Limoges, 2024–2025).
2. **OpenGL 4.5 Reference Pages**
Khronos Group. *OpenGL 4.5 API Documentation*.
<https://registry.khronos.org/OpenGL-Refpages/gl4/>
3. **GLSL Reference**
OpenGL Shading Language 4.5 – Specification & Quick Reference Guide.
https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language
4. **Assimp Library**
Open Asset Import Library (Assimp) – documentation technique pour le chargement de modèles 3D.
<https://github.com/assimp/assimp>
5. **GLM – OpenGL Mathematics**
GLM : Mathematics for 3D Graphics Programming – Documentation GLM.
<https://github.com/g-truc/glm>
6. **TheCherno – Modern OpenGL Series (C++)**
Série de tutoriels vidéo sur la création d'un moteur 3D avec OpenGL moderne (VAO, shaders, pipeline, etc.).
YouTube, playlist : https://www.youtube.com/playlist?list=PLlrATfBNZ98foTJPJ_Ev03o2oq3-GG0S2
7. **Dépôt GitHub du projet :**
https://github.com/Amaboss10/TP_M3D_ISICG/tree/newVersion